Laboratory 5
Subway model

In this laboratory we will create a model simulating passenger flow in the subway entrance hall. To get to subway trains passengers need to go through fare gates. Some passengers may have their tickets already bought earlier to avoid frustration queuing while their train is about to depart. Those may directly go to trains. Passengers without tickets need to buy tickets at the ticket office and only then may go through fare gates. This model demonstrates how to simulate pedestrian flow and services using AnyLogic Pedestrian Library.

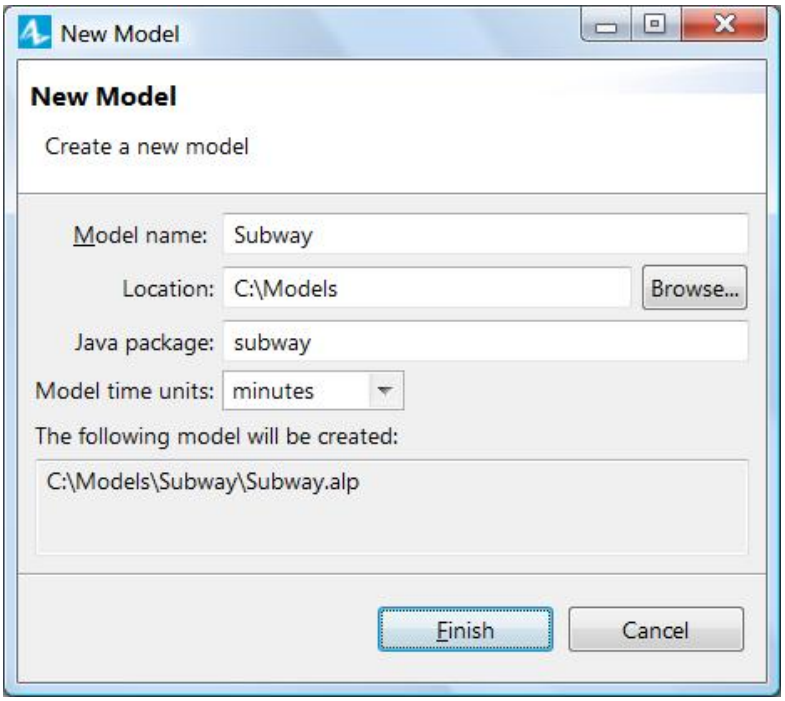
Please refer to Pedestrian Library Reference Guide for more information about Pedestrian Library blocks and their functions and parameters.

Phase 1. Simulating a simple pedestrian flow

In the first phase, we will create a simple model simulating a pedestrian flow: how people move inside our building.

1. Create a new model by selecting **File** > **New** > **Model** from AnyLogic's main menu. The **New Model** wizard will open.

2. Specify the name of the new project. In the **Model name** edit box, type: *Subway*.



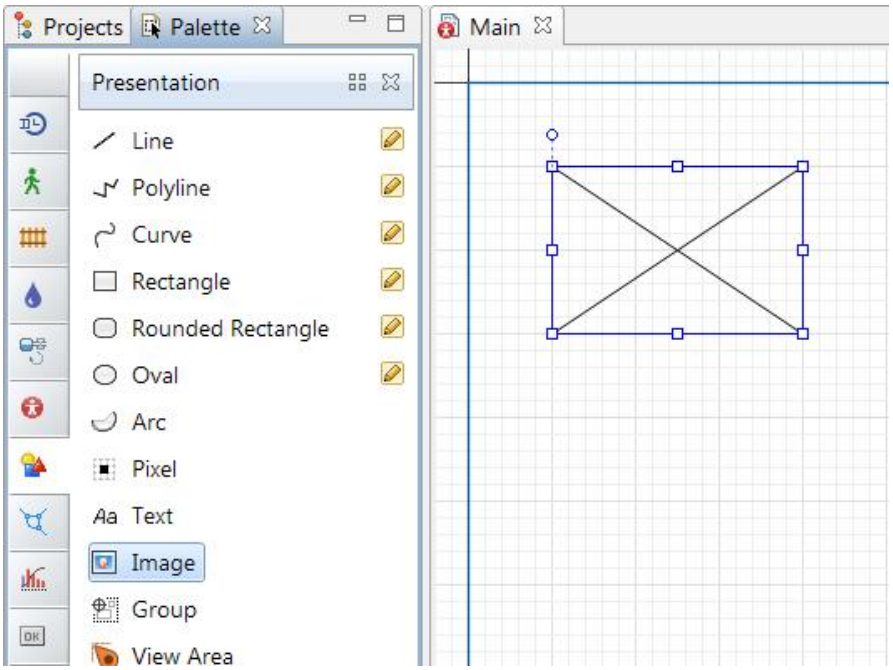
3. Specify the location where you want to store your model files. Browse for the existing folder using the **Browse** button, or type the name of the folder you want to create in the **Location** edit box.

4. Select *minutes* as the **Model time units**.

5. Click **Finish** to complete the process.

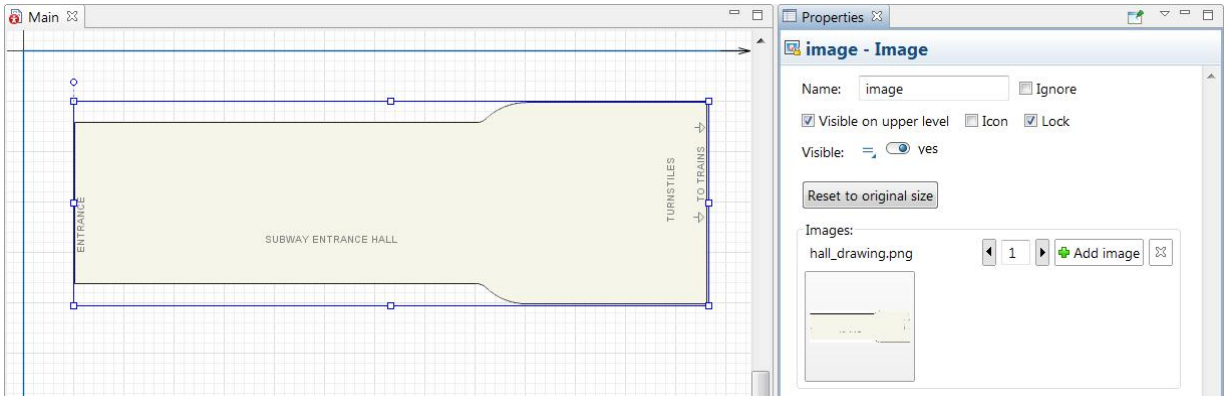
   When creating a pedestrian dynamics simulation model, you always start with adding a drawing of the simulated space. Then you draw the walls on top of this image and define the process logic: how the pedestrians move inside this building.

6. Drag the **Image** element from the **Presentation** palette on the graphical diagram. It will appear on the diagram as the crossed rectangle, see the figure below:



7. Now we need to choose the drawing file to be displayed by this image. The file browser will open automatically. Browse the folder where the file is located and then click **Open** in the dialog window. Otherwise, you can do this in the element's properties view. Open the image **Properties** view. Click **Add Image** button and navigate to the drawing file. You can find the drawing online.

8. You will see the drawing displayed in the graphical editor:

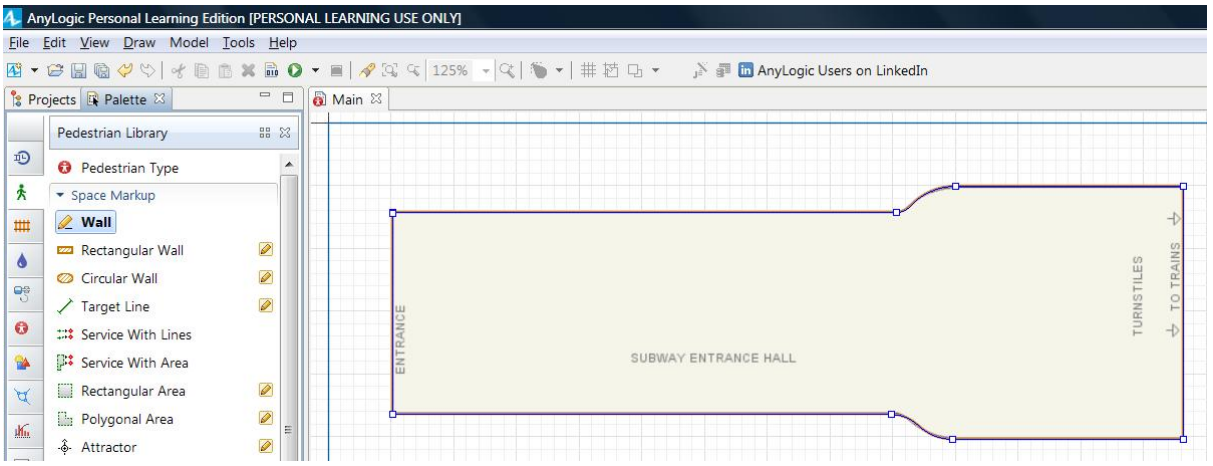


9. In the **Properties** view, select the **Lock** checkbox to lock the image. Locked shapes do not react on mouse clicks - it is impossible to select them in the graphical editor until you unlock them. We need this since we will draw other shapes on top of this image and thus want to prevent editing the drawing while drawing shapes over it.

   Just few zones are marked on this drawing. We want to experiment with different layouts and do not know right now where the ticket offices and ticket vending machines area will be located. That is the reason these areas are not marked on the drawing.

   Now we will mark up the space of our model. Usually we start with drawing the building walls.

10. Open the **Pedestrian Library** palette to draw the wall like in the figure shown below:



11. You draw the wall this way. First, double-click the **Wall** element in the **Space Markup** section of the **Pedestrian Library** palette. The icon of the element should turn into . It means that the *drawing mode* is activated and now you can draw the wall in the graphical editor point by point.

12. Subsequently click at the points where you want to place wall corners. Every click appends a linear segment to the wall you are drawing.

13. To append a curved segment, press the left mouse button at the point where the curved segment ends and then move the mouse with the button being pressed. While moving you will see how the curving radius changes. To draw the circular segment, move the mouse exactly along the grid line. When done, release the mouse button.

14. Finally double-click to place the last point and finish drawing the wall.

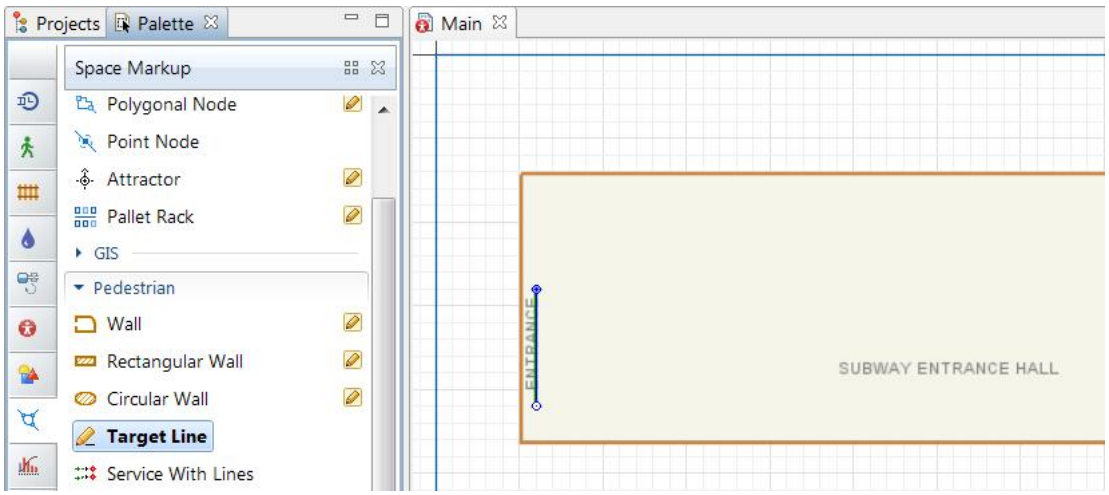
    If it is hard for you to draw a wall of exactly the same shape, once, you can draw the wall with several **Wall** shapes.

    Now you should define entry and exit points for the pedestrian flow. Draw an entry line first. Entry is a place where pedestrians appear in the simulated environment. You define the start line for the pedestrian flow with a special markup element **Target Line**.

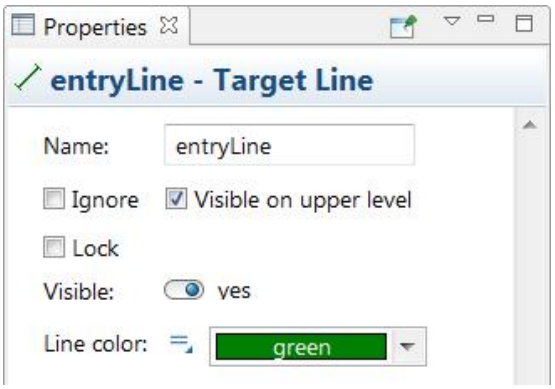
    We want to draw the entry line right where we have the ENTRANCE text on our drawing.

15. Drag the **Target Line** element from the **Space Markup** section of the **Pedestrian Library** palette onto the graphical editor.

16. Resize it and place it exactly as shown in the figure below:



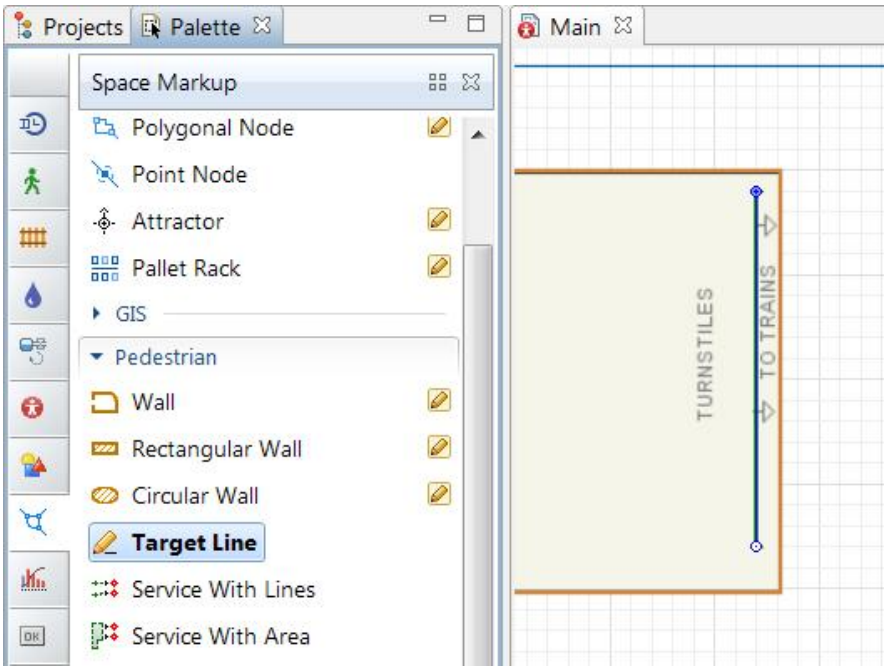
17. Name the line *entryLine*.



Now let's add one more target line that will define the place where the passengers will move when entering the subway entrance hall.

We want them to move to subway trains, so let's place this target line just over the text TO TRAINS.

18. Draw one more target line and place it as shown in the figure below. Entered the subway station, passengers will move here to get to subway trains.



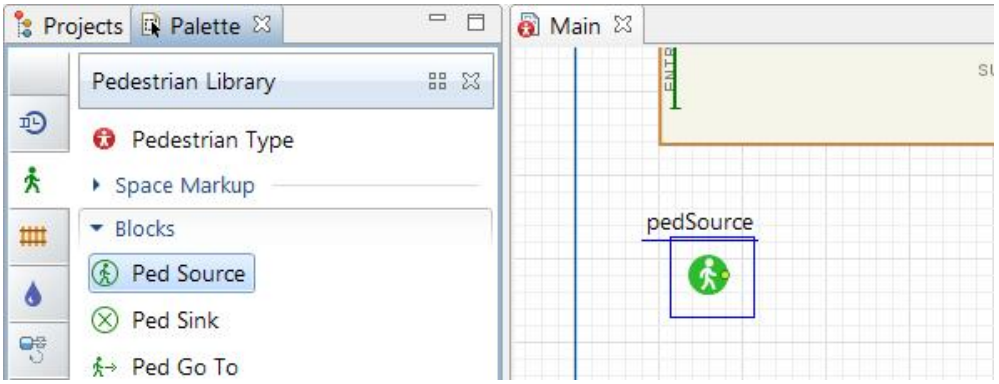
Note that all markup elements (target lines, etc.) should lie inside the walls shape.

Now we will finish creating the simple model simulating passenger flow. We will define the pedestrian flow process with a flowchart composed from **Pedestrian Library** blocks.

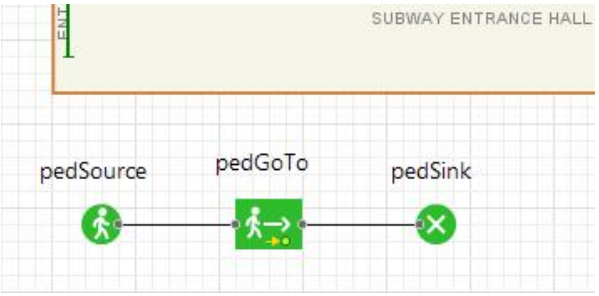
We will start with a very simple process: passengers enter the subway station (just where we put the *entryLine*) and then move to the trains (to our *targetLine*).

In AnyLogic you create flowcharts by adding the objects from the library palette to the graphical diagram, setting custom properties for the blocks, and connecting blocks together.

19. Add **Pedestrian Library** objects on the diagram and connect them as shown in the figure below. To add a flowchart block on the diagram, open the **Pedestrian Library** palette, and drag the required object from the palette into the graphical editor.



20. While dragging the blocks, and placing them close to each other, you may see the lines connecting the blocks appear. Please note that these connectors should connect only the ports lying on right and left borders of the block icons. This is critical, e.g. connecting the *pedSink* port to the lower port of *pedGoTo* block will raise the error.

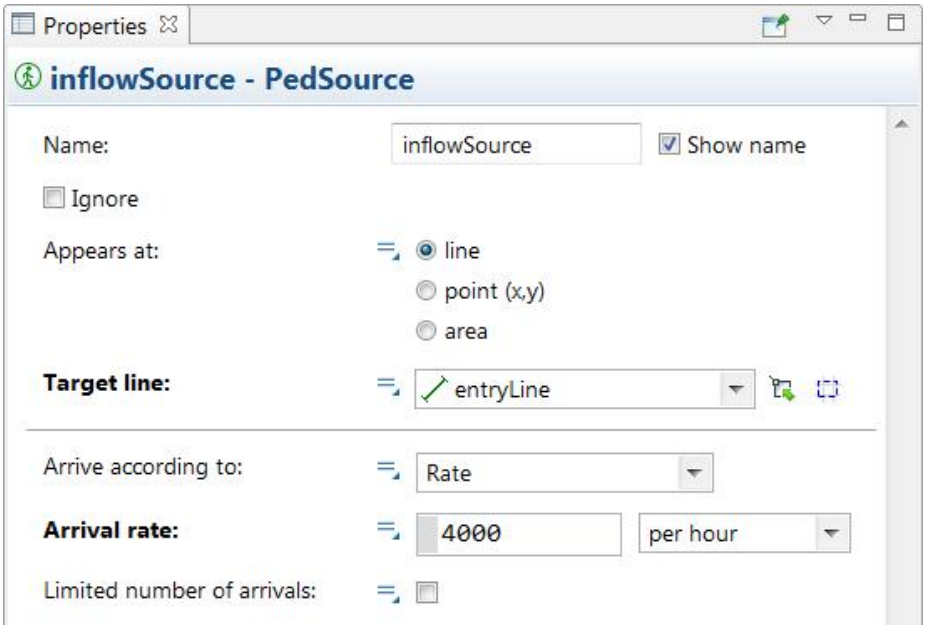


21. Rename the blocks. Name them *inflowSource*, *goToTrains*, *inflowSink*. (You can do this in the blocks' properties.)

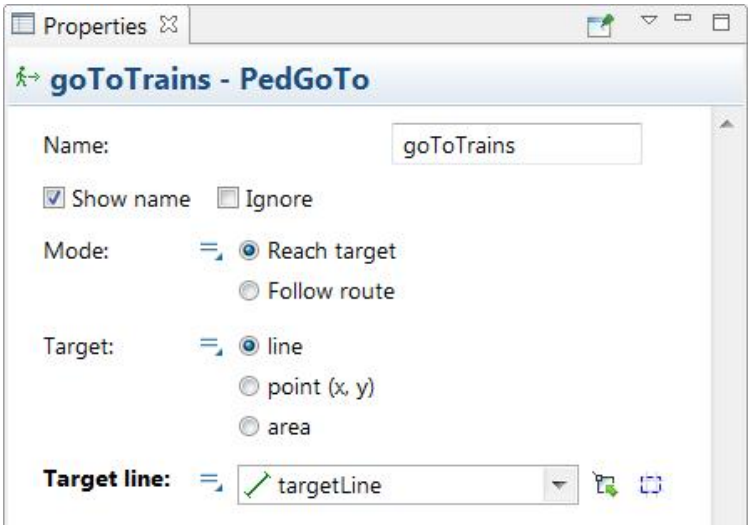


22. Select *inflowSource* block. In the **Properties** view, define the place where the passengers will appear. Choose *entryLine* (the name of our target line drawn earlier at the entrance) from the **Target line** drop-down list. Another way to choose a markup element in the parameter is to click the selection button and select this element in the graphical editor.

23. Set passengers **Arrival rate** to *4000 per hour*.

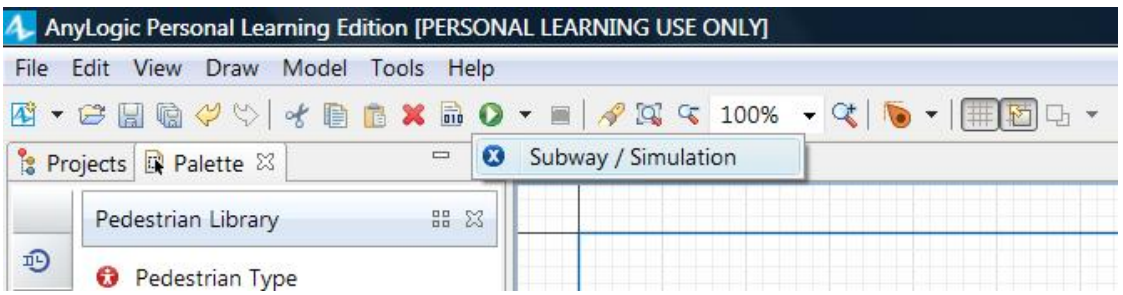


24. Now modify the properties of *goToTrains*. Specify destination for passengers. When entered the building, pedestrians will go to the target you specify here. At this point we assume that all passengers move directly to subway trains. Specify *targetLine* (the name of the target line we have drawn second) in the **Target line** field.

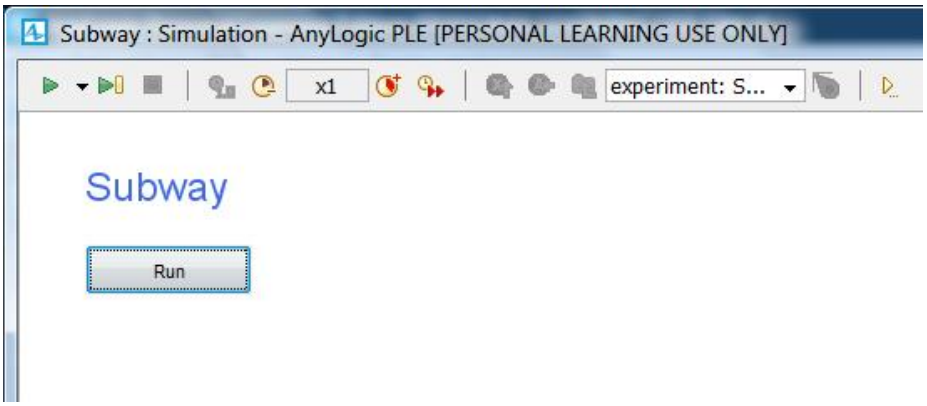


25. Leave all properties of **PedSink** object by default.

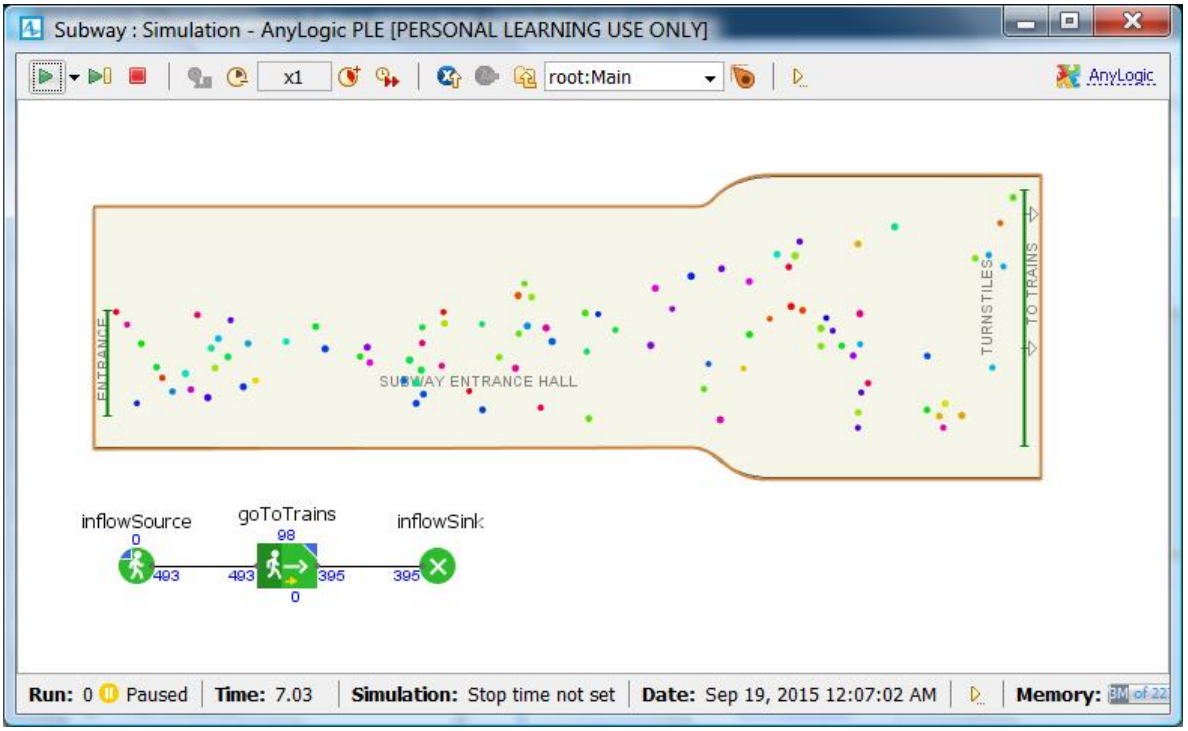
26. Click the **Run** toolbar button and choose the experiment you want to run from the drop-down list. Your simulation experiment is called **Subway/Simulation**.



Having started the model, you will see the presentation window. It displays the presentation designed for your simulation experiment. By default it is blank, with just the model name and the button. Later on you can add here graphics, put here the model description, etc.



27. Click the button to **Run** and observe the process you have modeled. You can see passengers entering the subway entrance hall and passing through the corridor to the subway trains.

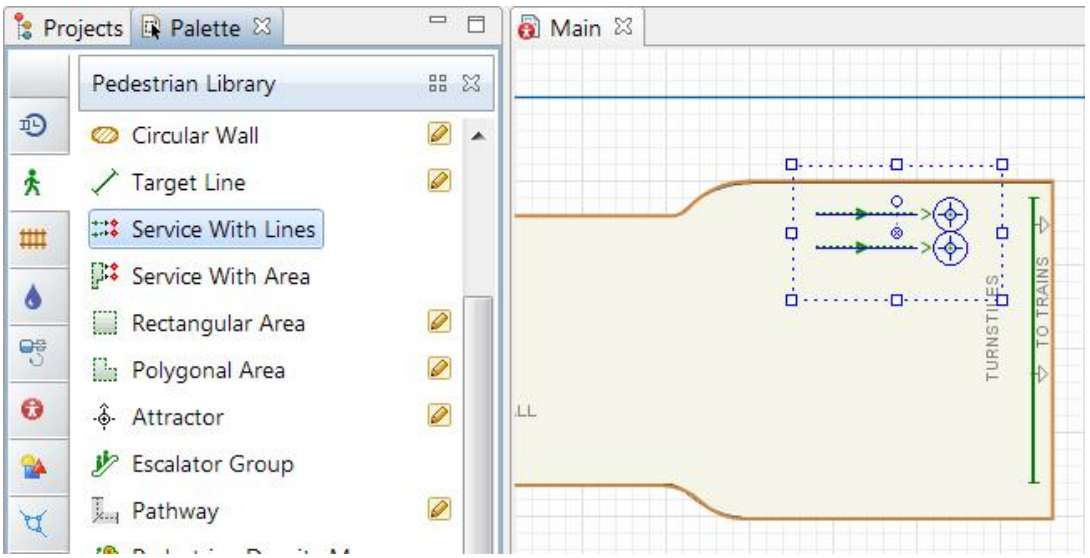


Phase 2. Adding fare gates

In the first phase, we created the model simulating a simple pedestrian flow: passengers enter the subway entrance building and go to the trains walking through the hall.

Now we want passengers to pass through the fare gates before entering the subway platform. So let's add fare gates at the end of the corridor.

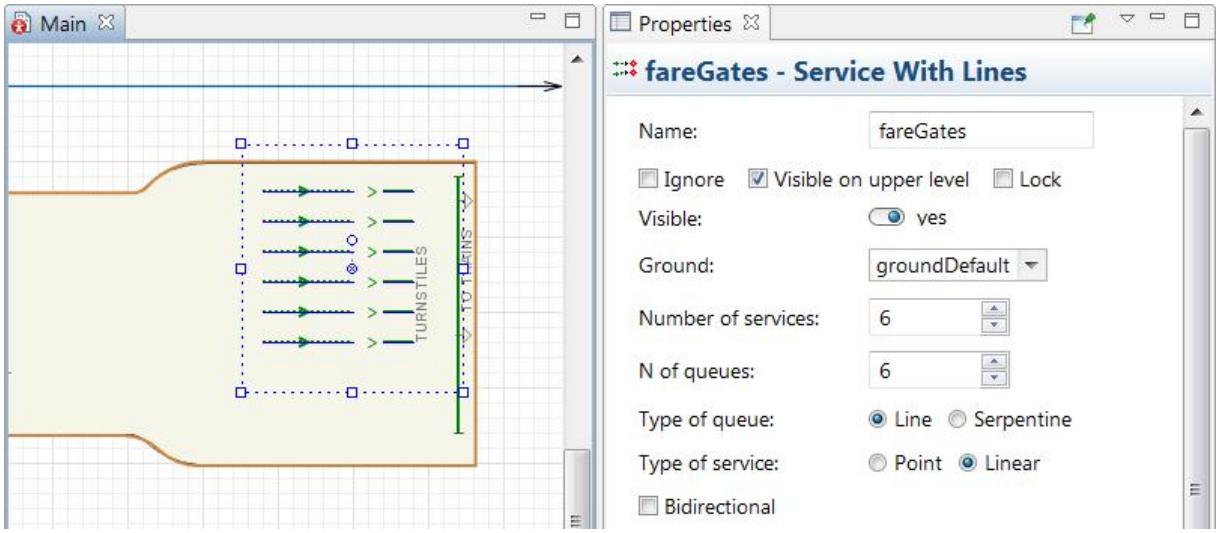
1. Drag the **Service with Lines** element from the **Space Markup** section of the **Pedestrian Library** palette into the graphical editor. You will see two service points and two queue lines leading to these points. Place the shapes as shown in the figure below:



2. Configure the services. Name it *fareGates*.

3. It's evident that two fare gates are not enough. Increase the **Number of services** to *6*. Symmetrically, change the **N of queues** also to *6*.

4. Change the **Type of service** from **Point** to **Linear**.



Now we will modify the model flowchart.

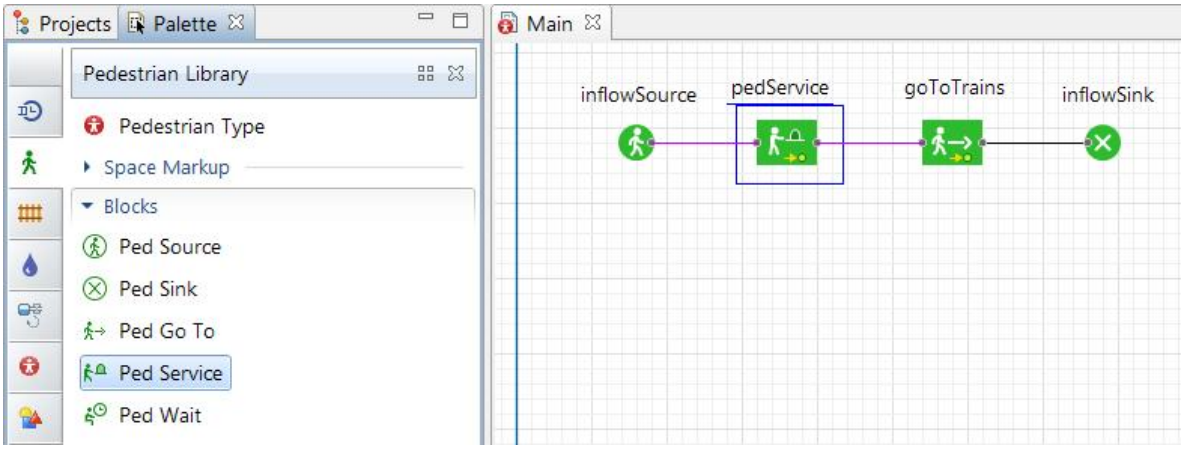
5. Prepare the place for the new block in our flowchart. Move *goToTrains* and *inFlowSink* blocks to the right as shown below:



6. Now we can insert **PedService** block in our flowchart. **PedService** simulates how pedestrians go to the services defined graphically with the markup shape and get serviced there.

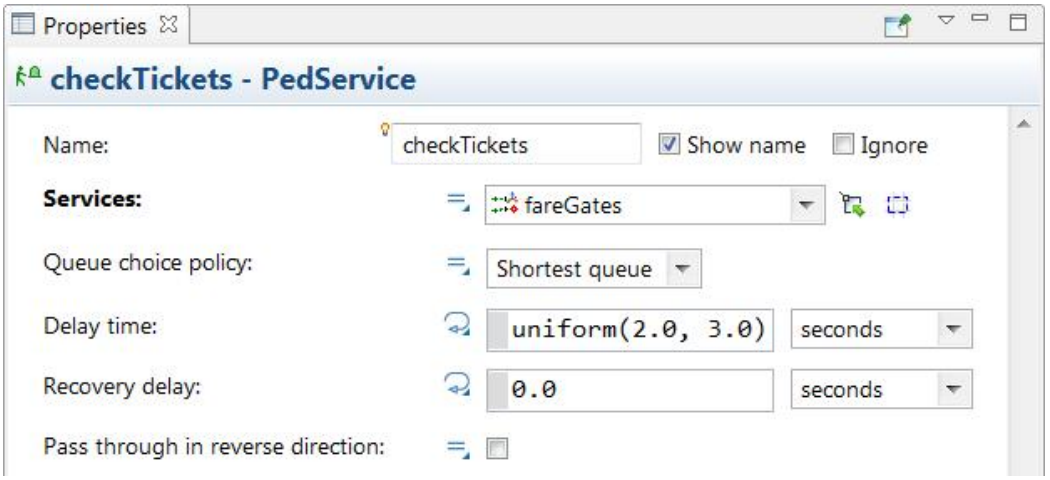
Drag **PedService** block from the **Pedestrian Library** palette onto the graphical diagram, right after the pedestrian source block. Placing the block right onto the connector, we connect the ports automatically (but please note that the right port of the *pedService* block should get connected, not the lower port).

7. Now *pedService* will appear in the flowchart, connected automatically.



When selecting the connector by the mouse click, its end point should be highlighted with cyan. This indicates the correct connection. In the case you do not see a cyan circle inside the port, it means that you put connector's point close by a port and you need to move the connector's end point exactly on the port.

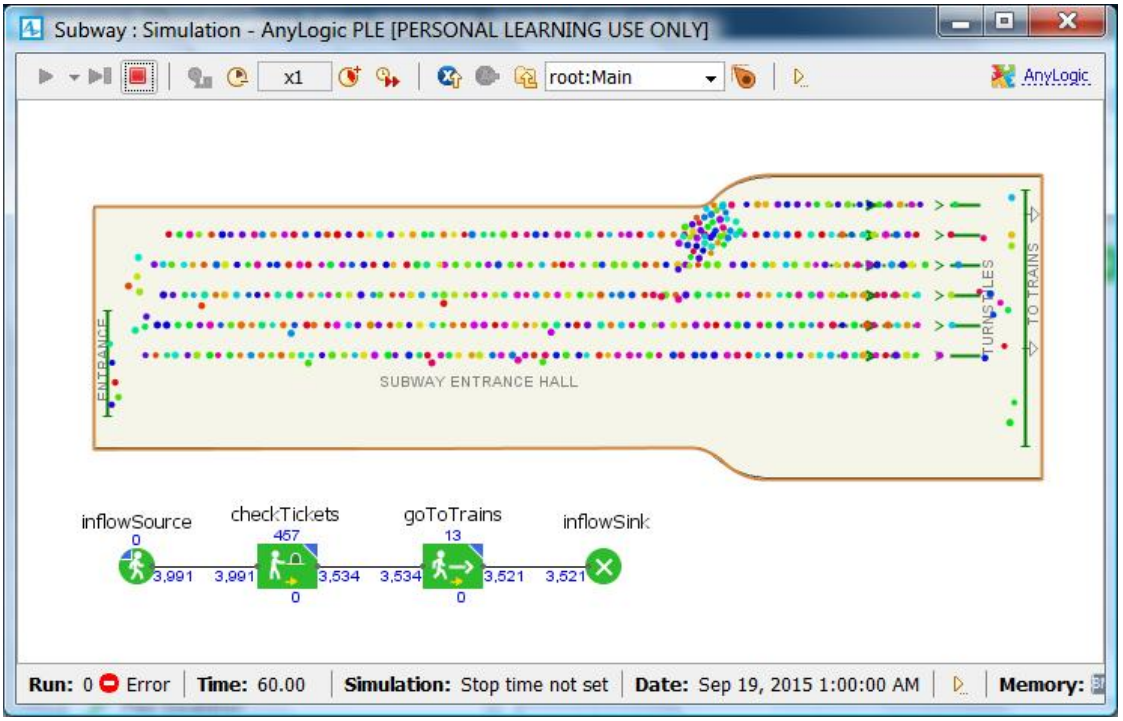
8. Open the **Properties** of the *pedService* block.
9. Rename the block to *checkTickets*.
10. Choose *fareGates* (the name of our **Service with Lines** markup shape) as the **Services**.
11. You can see that the **Delay** time is uniformly distributed with minimum value of *2.0* seconds and maximum value of *3.0* seconds. Leave it since it is typical delay time for fare gates.



We have defined the new logic and now we can run the model and observe its behavior.

After the model is successfully built, you can start it. Running the simulation, you automatically bring the current model up to date.

12. Run the model. You can see that now passengers pass through fare gates and the lines get very long instantly. So we should increase the number of fare gates.



6 fare gates. Terrible lines in seconds.

13. Stop the model and open the properties of the service shape again. Increase the number of services and number of queues to *7*.

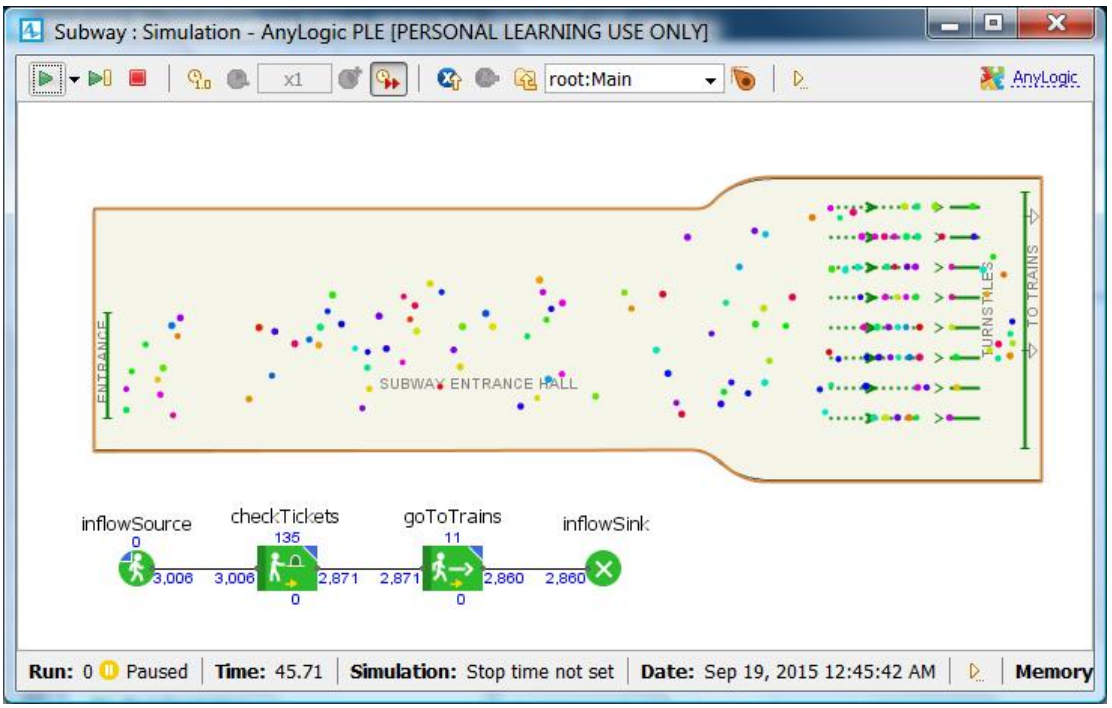
    Run the model again and see how the things are going on now. You may think that everything is OK now. But actually we simulate the model rather slowly, to understand the global trend, we recommend you to run the model faster to get the answer whether the lines are growing or not.

    You can adjust the execution speed using **Slow down** and **Speed up** toolbar buttons. Right now, it is better to switch to virtual time mode to view simulation run at its maximum speed and simulate a long period of time. To enable virtual time mode click the **Toggle real/virtual time** mode toolbar button.

    Simulating a long period of time gives us the answer. Lines are still growing, not so fast as before, but sometimes they may become rather significant. So, *7* fare gates probably are still not enough to serve the flow of 4000 pedestrians per hour. (Please remember that this applies to our fare gates with the service time of 2-3 seconds.)

    So let's increase the number of fare gates once more. Please change the number of services and number of queues to *8* and run the model again.

    Finally our configuration is acceptable - the station can cope with such pedestrian flow successfully.



8 fare gates. Queues are reasonable

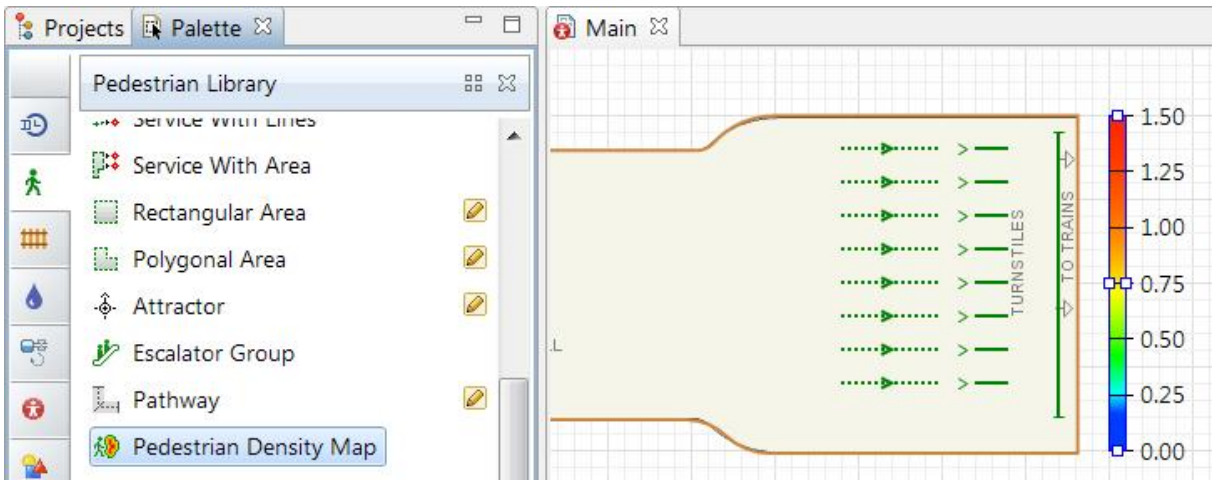
So we got the first practical use of our model. This model helped us to find the required number of the service points. You can play with pedestrian arrival rates in the settings of the **PedSource** block and analyze how to improve the facilities to cope with the load.

Phase 3. Displaying pedestrian density map

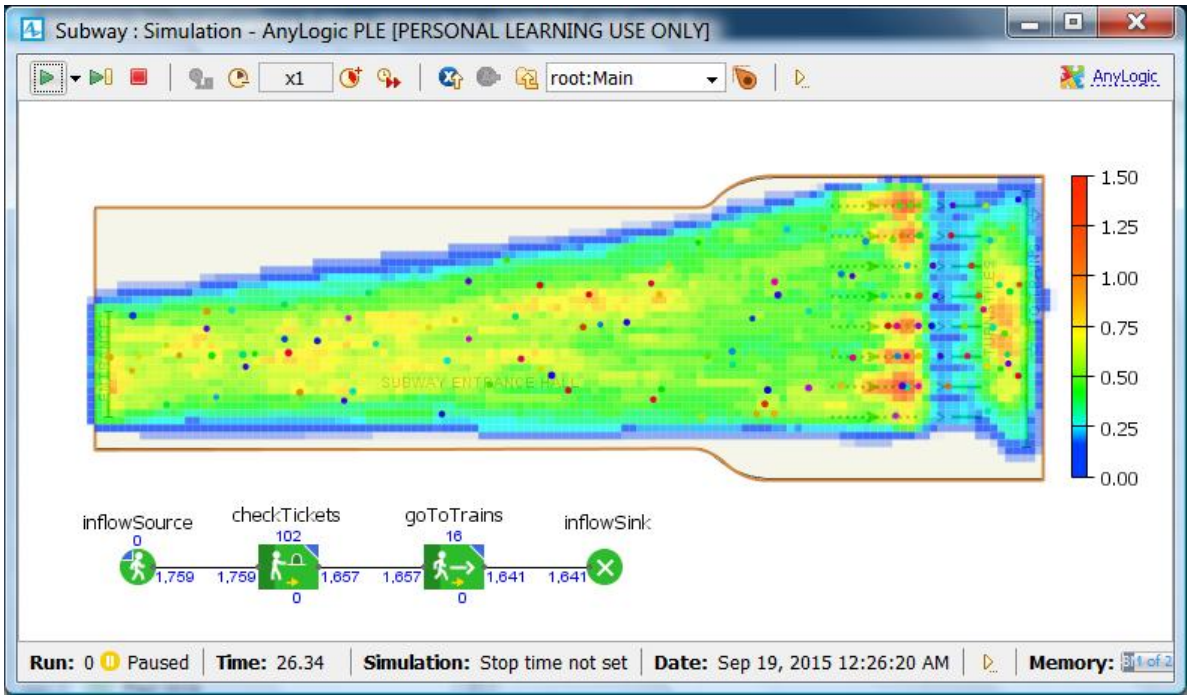
We have created the model simulating pedestrian flows inside the subway entrance building. Now we want to get some statistics on the flow. The most powerful tool used in pedestrian simulation is *Pedestrian density map*.

1. Drag the element **Pedestrian Density Map** from the **Space Markup** section of the palette **Pedestrian Library** into the graphical editor. The element acts as the color scheme. At the runtime it will show the density map legend. This density map legend helps to understand what colors on the density map mean what values. Legend displays the correspondence between density values and colors on the density map.

   Finally our configuration is acceptable - the station can cope with such pedestrian flow successfully.



Now you can run the model and observe its behavior.



You will see that as pedestrians move in the simulated space, the layout is gradually painted in different colors. The color of every point of the space corresponds to the current density in this particular area. The density map is constantly repainted according to the actual values: when the density changes in some point, the color changes dynamically to reflect this change. In case of zero density the area is not painted at all.

Phase 4. Adding ticket vending machines

At the moment all the passengers in our model enter the subway entrance building, then pass through the fare gates and go to the trains. So we assume that all passengers have bought their tickets in advance. Actually this can be hardly true. Some people may enter the station with the tickets being purchased in advance. But a lot of people buy tickets just when they enter the subway station.

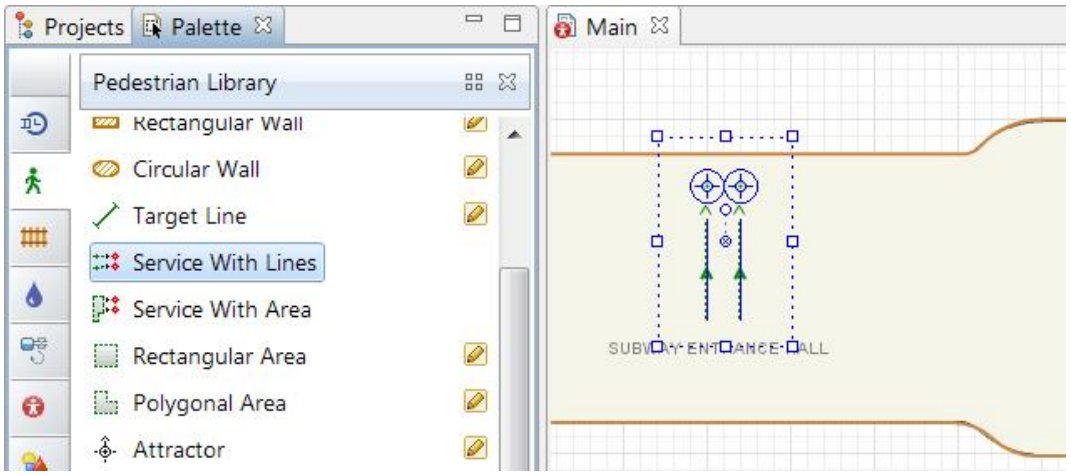
There can be different ticket vending facilities in the station. Small subway entrances may provide only ticket vending machines, big and spacious stations also have ticket offices.

Let's expand our model by adding ticket vending machines first. Having built such a model, we want to know the number of machines we need to satisfy the passenger needs and we can also find the optimal location for the machines to minimize the passenger flows intersections and crowds.

Like fare gates, ticket vending machines can be also naturally represented as **Services with Lines**.

1. Drag the **Service with Lines** element from the **Space Markup** section of the **Pedestrian Library** palette into the graphical editor.

2. Rotate the service shapes and place them near the wall as shown in the figure below:

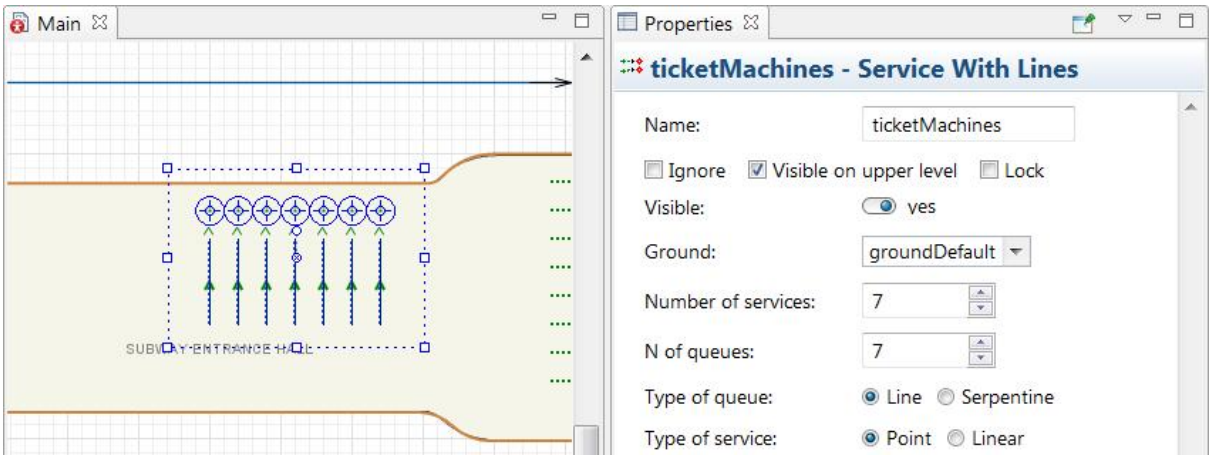


3. Open the services properties and configure this group of services.

4. This time our services are not linear, but point. Pedestrian just reaches the service point and stays there for service delay time. So leave the **Type of service** set to **Point**.

5. Name the services *ticketMachines*.

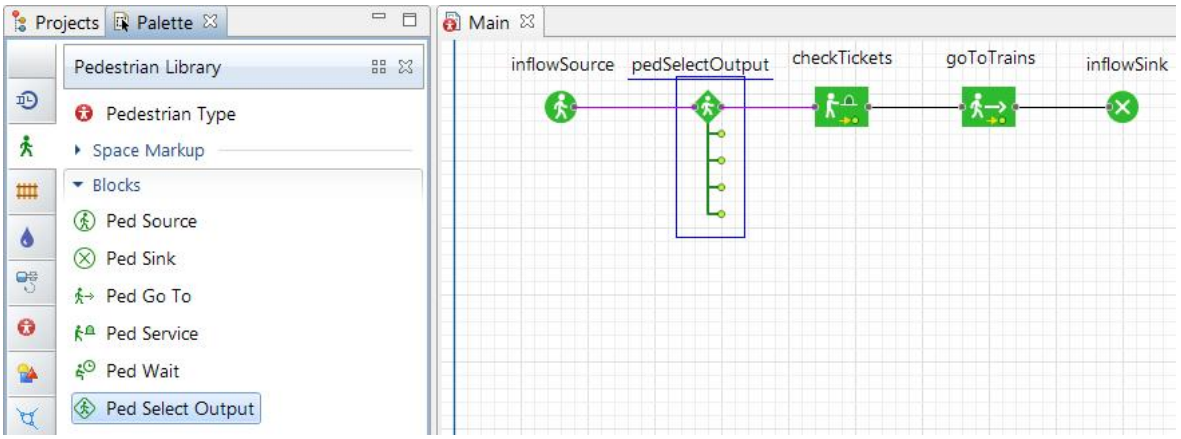
6. Increase the **Number of services** to *7*. Symmetrically, change the **N of queues** also to *7*.



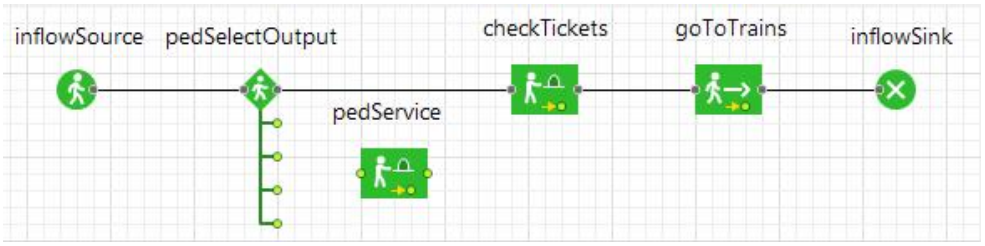
Now we want to route some passengers directly to fare gates, and some - to get serviced at the ticket vending machines.

7. Add **PedSelectOutput** block to separate passenger flows. We need this block to route non-ticketed passengers to ticket vending machines while other ones – straight to the fare gates. **PedSelectOutput** object is a decision making block of **Pedestrian Library**. Pedestrians arriving at the **PedSelectOutput** object are forwarded along one of its output ports depending on the specified ratios.

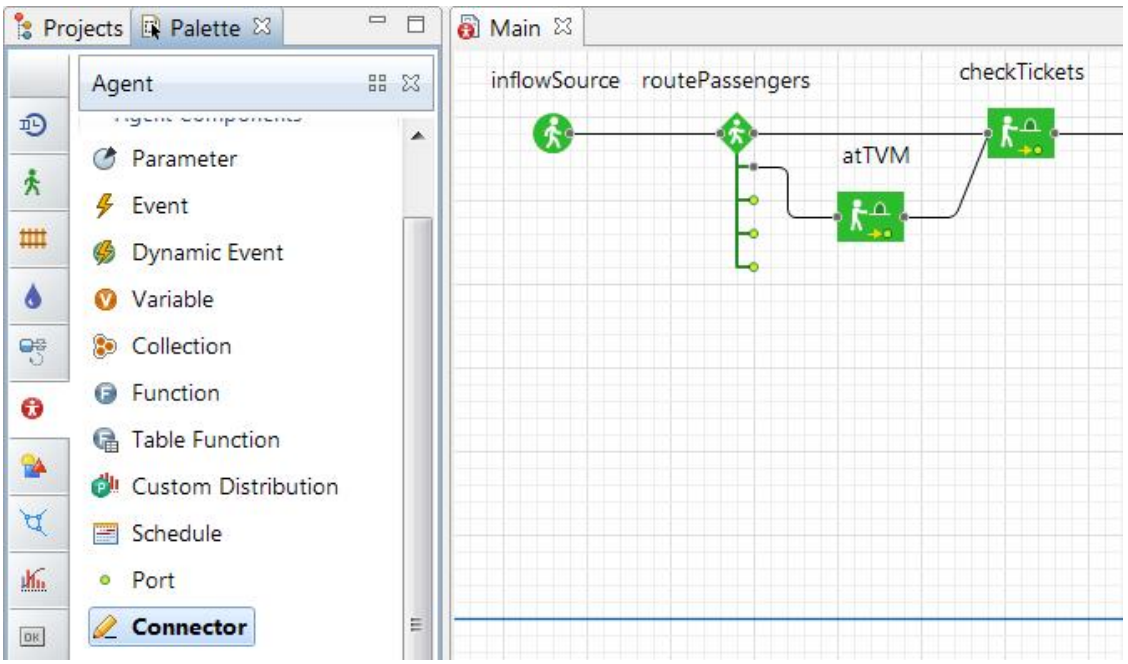
8. Move the blocks *checkTickets*, *goToTrains* and *inflowSink* to the right to make space for the new block in the diagram between *inflowSource* and *checkTickets*. Then insert **PedSelectOutput** in the resulting space. The block will be automatically connected when you place in on the connector.



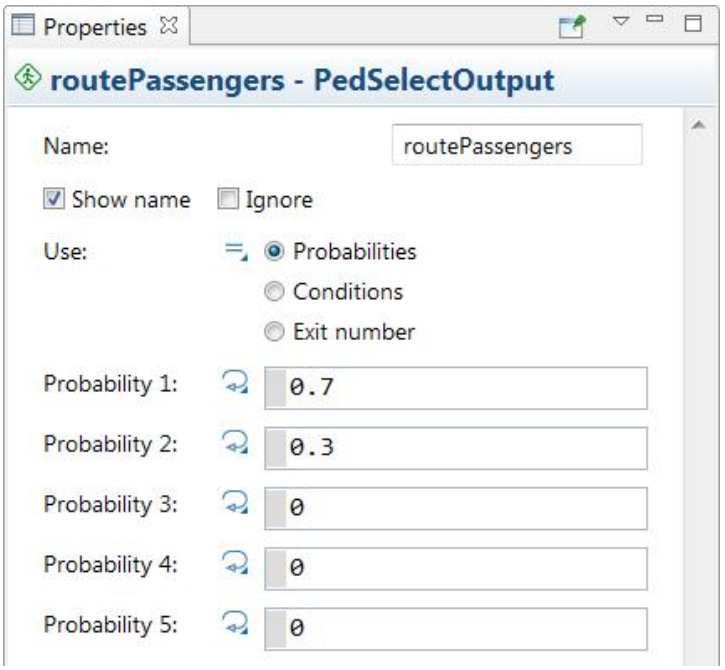
9. Add one more **PedService** block. This block will simulate how passengers are serviced at ticket vending machines. Place it between the second *out* port of **PedSelectOutput** and the existing **PedService** block (*checkTickets*).



10. Connect the block as shown in the figure (this will require drawing two new connectors between the second *out* port of **PedSelectOutput** and *checkTickets* library blocks).



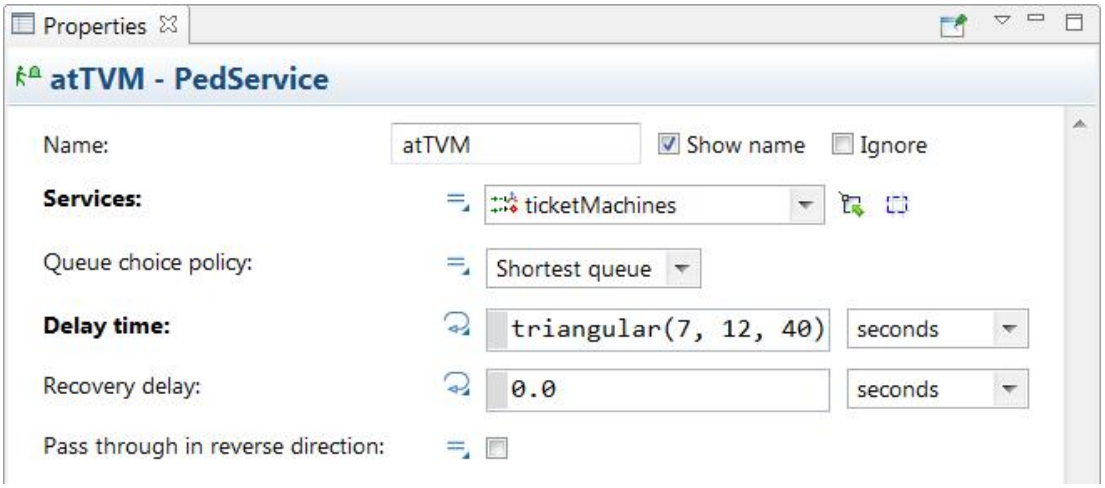
11. Configure the **PedSelectOutput** block. Name the block *routePassengers*. Specify *0.7* as **Probability 1** (the ratio for flow going directly to fare gates), and *0.3* as the **Probability 2** (ratio for the flow going to ticket vending machines correspondingly. In this flowchart we assume that the number of passengers that have already bought their tickets is actually significantly higher. Set **Probability 3, 4, 5** equal to *0*.



12. Configure the just added **PedService** block. Rename the block to *atTVM*.

13. Choose *ticketMachines* (the name of our **Service with Lines** markup shape) as the **Services**.

14. Change the **Delay** time. Write there: *triangular(7, 12, 40)* and choose *seconds* from the drop-down list of units. We assume that service time is triangularly distributed with minimum value of 7 seconds, average value of 12, and maximum value of 40 seconds.



Let's run the model and observe its behavior. Note that now some passengers first go to the ticket vending machines to buy a ticket and only then pass through fare gates.

